

Preloading

Overview

Students have called `this.load.image` since Chapter 1 without thinking about it. This chapter opens the loader up: `load...` calls only **queue** files; the loader fetches them after `preload()`; loaded files live in **game-wide caches** under **keys**. On that foundation students build the standard **Boot -> Preload -> Menu** structure with a progress bar (their first use of `Graphics`), load every common kind of file (including JSON described by an interface, and an asset pack), learn what a missing file looks like, and load files in the middle of a game with `this.load.start()`.

The practical difficulty of teaching it: **local files load too fast to see**. The loading screen project pads its queue with 150 copies of small pictures, but the honest demonstration is the browser's network throttling - rehearse it before the session.

Prerequisites

- Chapter 1: building and serving, config/scene/game object, `preload()` / `create()`
- Chapter 2: several scenes, `this.scene.start(key, data)`, `init(data)`, interfaces, `on / once`, spread
- JSON as a format (most students have met it; a two-minute reminder is enough)

Learning outcomes

Students can:

1. explain what `this.load.image(key, file)` does and does not do, and when `create()` runs
2. say where loaded files are kept, and use keys to read them back from `this.textures` and `this.cache.*`
3. structure a game as Boot -> Preload -> Menu, and explain why the boot scene exists

4. draw a progress bar with `Graphics` , driven by the loader's `"progress"` event
5. load images, sprite sheets, audio, JSON (typed with an interface), text and asset packs
6. recognise and diagnose a missing file (the missing texture, `"loadererror"` , the console's 404)
7. load files after `preload()` with `this.load.start()` and `"complete"` , checking the caches first

Suggested session plan (2 hours)

Time	Activity
0:00 - 0:10	Ask: "what happens between pressing refresh and seeing the title screen?" Collect answers; then show the Network tab of the developer tools while <code>ch02_click_the_ball_plus</code> loads
0:10 - 0:25	Slides 3-6: the queue, keys and caches, paths, <code>setPath</code> . Draw the queue -> loader -> caches diagram
0:25 - 0:45	Slides 7-11: Boot -> Preload -> Menu, <code>Graphics</code> , loader events. Walk through <code>PreloadScene.ts</code> . Live demo: run it normally (a flash), then with throttling on "Fast 4G"
0:45 - 1:00	Challenges 1 and 2
1:00 - 1:20	Slides 12-16: every kind of file, JSON and interfaces, asset packs, missing files. Live demo: rename a picture in <code>public/</code> , rebuild, find it with the console
1:20 - 1:30	Slides 17-18: loading later. Play <code>ch03_load_on_demand</code> throttled; press U and go back in
1:30 - 2:00	Challenges 3-6

For a 1 hour lecture + 2 hour lab: lecture slides 1-18 with the three demos; lab challenges 1-6.

Key points to stress

- `preload()` **only makes a list.** Nothing is loaded until it returns. This one sentence explains most loading bugs students write
- **The caches are the game's, not the scene's.** A file loaded anywhere can be used everywhere, and a key that is already loaded is skipped. This is what makes both "load everything at the start" and "load on demand, but only once" work
- **Keys are names; files are addresses.** After `preload()` the file name should never appear again. Keep keys as constants, as scene keys were in Chapter 2
- **Paths are relative to the page** (`dist/index.html`), not to the `.ts` file - and everything in `public/` is copied to `dist/`, so "not in `public/` " means "not there"
- **The boot scene exists because a loading screen cannot show something it is still loading**
- **Outside `preload()` , you start the loader yourself.** No `this.load.start()` , no loading, no error
- `cache.json.get` returns `any` ; the interface is a promise *about* the file, checked against the code, never against the file. Good discussion of what static typing can and cannot do

Common problems and errors

What students see	Cause	Fix
A black square with a green outline and a green diagonal	the missing texture: no picture with that key	check the console for a 404; check the file is in <code>public/</code> , and the name, folder and capitals; check the key matches
Works at home, missing pictures once uploaded (or on a lab machine)	capitals: <code>Sky.png</code> vs <code>sky.png</code> - macOS and Windows usually do not mind, web servers do	make the names match exactly; use lower case for every file
Error: Audio key "shoot" not found in cache in the console, and the scene stops	<code>this.sound.play(key)</code> for a sound that is not loaded (misspelt key, missing file, or loaded in a scene that has not run yet)	load it first; check the key
Error: TextureManager.SpriteSheet: Invalid frameWidth given. and an empty screen	<code>this.load.spritesheet(key, file)</code> with no <code>{ frameWidth, frameHeight }</code>	give the frame size (the asset library README lists them)
A sprite sheet's frames are cut in the wrong places, or show parts of two pictures	wrong <code>frameWidth / frameHeight</code>	check the frame size
Parameter 'value' implicitly has an 'any' type. on a loader listener	the listener's parameter has no type; <code>this.load.on</code> accepts any function, so TypeScript cannot work it out	write the type: <code>(value: number)</code> , <code>(file: Phaser.Loader.File)</code>
Property 'gem' does not exist on type 'LevelData'. Did you mean 'gems'?	a typo reading JSON data - the interface catching it	fix the name
Property 'hearts' does not exist on type 'LevelData'.	the JSON file gained a field; the interface did not	add it to the interface
Cannot read properties of undefined (reading 'length') in the browser	the interface has a field the JSON file does not (misspelt in the file, or a missing comma broke the file)	fix the file; check it is valid JSON

What students see	Cause	Fix
Loading in <code>create()</code> does nothing at all	forgot <code>this.load.start()</code>	call it after queueing
A new picture under an old key does not appear; the old one stays	the loader skips keys already in the cache	<code>this.textures.remove(key)</code> first, or use a new key
Loading screen shows the missing texture instead of the logo	the logo was loaded in <code>PreloadScene</code> , not <code>BootScene</code>	load what the loading screen shows in the boot scene
The bar never appears / the loading screen flashes past	local files are fast	throttle the network; that is the real test
A path starting with <code>/</code> works with <code>deno task serve</code> , then breaks elsewhere	<code>/</code> means the top of the web server, not the page	leave the leading <code>/</code> off

A subtle one for the curious: in `preload()`, `this.time.now` can be out of date - the scene's clock is only updated once the scene is running frames, and `preload()` comes before its first frame. The first time a scene runs it holds the time the game started (close enough, for a scene that starts straight away); when the scene is started again it holds the time the scene last stopped. That is why `PreloadScene` times the load with `performance.now()`. Chapter 4 explains the order in which Phaser does things.

Suggested demos

- **The queue is not the load.** In `BootScene.preload()`, add `this.add.image(400, 300, LOGO_KEY)` straight after `this.load.image(LOGO_KEY, ...)`. The missing texture appears (briefly - the scene moves on). Ask why, before explaining
- **Throttling.** Run `ch03_loading_screen` normally, then with the Network tab's "Disable cache" and "Fast 4G". Show the waterfall of requests: several at once, big files taking longest. Then set `COPIES` to 0 and run it again throttled - the bar still moves, because the music is big
- **Loading again.** Press R in the menu: "Loaded 0 files". Ask what the loader did with 170 keys that were already loaded

- **A missing file.** Rename `sky.png` in `public/assets/images/`, rebuild, refresh. Find the 404 in the console; notice the rest of the game still works. Then misspell a sound's key and watch the scene stop with an error instead
 - **Unloading.** In `ch03_load_on_demand`, visit Space, press U, visit Space again with throttling on: it loads again. Discuss what a 40-level game would do
-

Discussion questions

- A loading screen is a scene that shows pictures while loading pictures. How does the boot scene get round this? What else would you put in a boot scene?
 - Why does the loader skip a key that is already loaded? When is that helpful, and when is it a trap?
 - JSON level files, or levels written in TypeScript? What does each make easier? Who else might edit a JSON file?
 - An asset pack moves the list of files out of the code. The code still needs the keys. What has been gained?
 - Load everything up front, or on demand? Think of a game where each is clearly right. (A small arcade game; an open world, or a game with many levels.)
 - `this.textures.remove(key)` while a game object is still showing that picture - what would you expect? Why is it safe in the rooms project?
-

Extension ideas

- load a **bitmap font** or a **web font** (`this.load.font`) and use it on the menu
- a loading screen whose tips change: load `tips.txt` in the boot scene, split it into lines with `split("\n")`, and show a random one under the bar
- `filecomplete-json-level`: load a JSON file that *lists* other files, and queue those from inside that event while the loader is still running (the progress bar goes backwards - why?)
- preload the *next* room in the background while the player is in the current one
- count bytes rather than files: add up `file.bytesTotal` for a truer progress bar

Assessment ideas

- Practical: "add a fifth room with its own background and three pictures; nothing should need to change except `rooms.ts` and `public/`"
- Code reading: show a `create()` that queues two files and adds images of them straight away, with no `start()` and no `"complete"`. What does the player see? Fix it
- Short answer: explain the difference between a key and a file name, and why the rest of the game should only use keys
- Short answer: what is the missing texture, and name three different causes of it

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment (JSON files cannot have comments, so changes to them are described here instead).

To see a loading screen solution working, slow the network down (Chrome or Edge: F12, **Network, Disable cache, Fast 4G**) - or it is over before you can look at it.

1. Restyle the bar

Project: `solutions/ch03_challenge_1_restyle_the_bar` (from `ch03_loading_screen`)

Goal: find where each part of the loading screen is decided, and learn a little more of the `Graphics` API.

`src/scenes/PreloadScene.ts`

```
// CHALLENGE 1: a longer, taller bar, in a rounded box with an outline
const BAR_WIDTH = 500;
const BAR_HEIGHT = 36;
const BAR_Y = 350;
const BOX_COLOUR = 0x0b0e14;
const BOX_OUTLINE = 0xf1faee;
const BOX_PADDING = 8;           // the gap between the box and the bar
inside it
const BOX_RADIUS = 12;
```

```
box.fillStyle(BOX_COLOUR);
box.fillRoundedRect(boxX, boxY, boxWidth, boxHeight, BOX_RADIUS);
box.strokeStyle(3, BOX_OUTLINE);
box.strokeRoundedRect(boxX, boxY, boxWidth, boxHeight, BOX_RADIUS);
```

```
bar.fillStyle(this.barColour(value));    // CHALLENGE 1
```

```
// CHALLENGE 1: red, then orange, then green, as the bar fills
private barColour(value: number): number {
  if (value < 1 / 3) {
    return EARLY_COLOUR;
  }
  if (value < 2 / 3) {
    return MIDDLE_COLOUR;
  }
  return LATE_COLOUR;
}
```

plus new wording and colours for the two lines of text. Because the whole bar is cleared and redrawn on every "progress" event, choosing its colour from `value` each time is all it takes - the bar turns wholly orange at a third, and wholly green at two thirds.

Look for: the sizes kept as named constants, and the box worked out *from* them (so changing `BAR_WIDTH` still centres everything); `LineStyle` before `strokeRoundedRect` (the stroke uses whatever line style was set last - forget it and there is no outline); the colour choice in a small method rather than inline in the listener. A student who makes the colour **blend** from red to green (`Phaser.Display.Color.Interpolate.ColorWithColor`) has gone further than asked - welcome it.

2. Percentage

Project: `solutions/ch03_challenge_2_percentage` (from `ch03_loading_screen`)

Goal: turn the "progress" value into something a person reads; think about rounding.

`src/scenes/PreloadScene.ts`

```
// CHALLENGE 2: the percentage, in the middle of the bar. Made AFTER the
// bar, so it is drawn
// on top of it
const percentText = this.add.text(centreX, BAR_Y + BAR_HEIGHT / 2, "0%", {
  fontFamily: "Arial",
  fontSize: "20px",
  color: "#ffffff",
  stroke: "#1b1f2a",
  strokeThickness: 4,
}).setOrigin(0.5);
```

```
// CHALLENGE 2: value is 0 to 1; floor, so it only says 100% when it really
// is finished
percentText.setText(`${Math.floor(value * 100)}%`);
```

- **order of creation is order of drawing:** text made before the bar `Graphics` would be hidden behind the bar as it grows. A good moment to say so (Chapter 4 covers draw order and depth)
- `Math.round(0.996 * 100)` is 100 - the screen would say 100% with a file still to come. `Math.floor` never over-promises
- the stroke makes white text readable over both the yellow bar and the dark box

Look for: the update inside the existing `"progress"` listener (a second listener is fine too); a whole number (students who write `value * 100` get `36.470588...%`); the text drawn on top of the bar. Showing "12 of 170 files" as well, with `this.load.totalComplete` and `this.load.totalToLoad`, is a good extension.

3. Hearts in the level

Project: `solutions/ch03_challenge_3_hearts_in_the_level` (from `ch03_asset_types`)

Goal: change a data file *and* its TypeScript description together.

`public/assets/data/level.json` - a new list, after `"gems"` (note the comma that now has to follow the `]` of `"gems"`):

```
"hearts": [
  { "x": 30, "y": 40 },
  { "x": 160, "y": 130 },
  { "x": 225, "y": 45 }
]
```

src/LevelData.ts

```
hearts: Point[];    // CHALLENGE 3: the new list in level.json
```

src/scenes/AssetTypesScene.ts

```
// CHALLENGE 3: say how many hearts too
this.add.text(x + 10, y + 34, `${level.name} - ${level.gems.length} gems,
${level.hearts.length} hearts`, SMALL_STYLE);
```

```
// CHALLENGE 3: the hearts - heart.png is already loaded, by the asset pack
level.hearts.forEach((heart) => {
  this.add.image(originX + heart.x, originY + heart.y, HEART_KEY);
});
```

No new file needs loading: `heart.png` comes in with the asset pack, and textures are shared, so the JSON panel can use it.

Look for: the interface updated. A student who skips it gets `Property 'hearts' does not exist on type 'LevelData'`. - which is the point of the interface. The opposite mistake - interface updated, JSON not (or the JSON broken by a missing comma) - builds cleanly and then fails in the browser: with no `"hearts"` in the file, `level.hearts.length` throws `Cannot read properties of undefined`. Ask students which of the two mistakes they would rather make. Sharp students make `hearts` optional (`hearts?: Point[]`) so older level files still work, and then have to check for `undefined` - a fine answer.

4. Keep it moving

Project: `solutions/ch03_challenge_4_keep_it_moving` (from `ch03_loading_screen`)

Goal: understand what runs while a scene is loading - and which scene must load what.

src/assets.ts

```
// CHALLENGE 4: the spinner is on the loading screen, so the boot scene must load it too  
export const SPINNER_KEY = "spinner";  
export const SPINNER_FILE = "assets/images/star.png";
```

src/scenes/BootScene.ts

```
this.load.image(SPINNER_KEY, SPINNER_FILE);    // CHALLENGE 4
```

src/scenes/PreloadScene.ts

```
const spinner = this.add.image(barX + BAR_WIDTH + 45, BAR_Y + BAR_HEIGHT /  
2, SPINNER_KEY);  
this.tweens.add({  
  targets: spinner,  
  angle: 360,  
  duration: SPIN_TIME,  
  repeat: -1,  
});
```

While a scene is loading, Phaser still runs its **systems** every frame - its clock, its tweens, its loader - but it does not call the scene's own `update()` until `create()` has run. So code in `update()` that turns the star does nothing until the loading is over; a tween turns it throughout. (Tested with the harness on a throttled network: the star's `angle` changes between two readings taken while `load.progress` is still below 1.)

`this.time.addEvent({ delay, loop: true, callback })` that changes the angle a step at a time also works, for the same reason.

Look for: the picture loaded in `BootScene`. Students who load it in `PreloadScene` see the missing-texture square spinning - worth letting them find that out and explain it; it is the whole reason for the boot scene. Students who try `update()` see a star that sits still until the bar is full.

5. When files fail

Project: `solutions/ch03_challenge_5_when_files_fail` (from `ch03_loading_screen`)

Goal: listen for failures, pass them to a new scene, and reset state for a scene that runs again.

`src/scenes/PreloadScene.ts`

```
private failed: string[] = [];    // CHALLENGE 5: every file that could not
be loaded
```

```
// CHALLENGE 5: the scene object is reused if the player tries again - start
with an empty list
init(): void {
    this.failed = [];
}
```

```
// CHALLENGE 5: "loaderror": one file could not be loaded - remember which,
and from where
this.load.on("loaderror", (file: Phaser.Loader.File) => {
    this.failed.push(`${file.key} - ${file.src}`);
});
```

```
// CHALLENGE 5: if anything failed, the error screen - it can still go on to
the menu
if (this.failed.length > 0) {
    const errorData: ErrorData = { failed: this.failed, menu: data };
    this.scene.start(ERROR_SCENE, errorData);
} else {
    this.scene.start(MENU_SCENE, data);
}
```

`src/scenes/ErrorScene.ts` (new)

```
export interface ErrorData {
  failed: string[];    // "key - url" for each file that failed
  menu: MenuData;     // what to pass on to the menu, if the player
                      carries on
}
```

```
this.input.keyboard!.once("keydown-R", () => {
  this.scene.start(BOOT_SCENE);
});
this.input.keyboard!.once("keydown-SPACE", () => {
  this.scene.start(MENU_SCENE, this.error.menu);
});
```

To test it, the solution asks for `strr.png` (a misspelling) when `TEST_A_MISSING_FILE` is `true`. Tested with the harness: the error screen lists `broken - assets/images/strr.png`; R runs the loading screen again and comes back to the error screen with **0** files loaded (everything that worked is already in its cache, so only the broken file is tried again); SPACE reaches the menu.

Look for:

- `init()` resetting the list - without it, each retry adds the same failure again, and the list grows (the Chapter 2 trap, in a new place)
- the decision made in `create()` (or on `"complete"`), after all files have been tried - not in the `"loaderror"` listener, which would leave while other files were still loading
- `file.src` shown, not just the key: the address is what tells you *why* it failed
- the answer to the hint's question: a retry only fetches what failed, because the rest is cached

6. Rooms from a pack

Project: `solutions/ch03_challenge_6_rooms_from_a_pack` (from `ch03_load_on_demand`)

Goal: separate *which files* (data, in a pack) from *what to do with them* (code); discover that a pack is itself a cached file.

public/assets/rooms_pack.json (new) - one section per room:

```
"space": {
  "path": "assets/",
  "files": [
    { "type": "image", "key": "space", "url": "images/space.png" },
    { "type": "image", "key": "player_ship", "url": "images/player_ship.png" },
  ],
  { "type": "image", "key": "enemy_ship", "url": "images/enemy_ship.png" },
  { "type": "image", "key": "rock", "url": "images/rock.png" },
  { "type": "audio", "key": "shoot", "url": "audio/shoot.wav" }
]
```

src/rooms.ts

```
export interface Room {
  name: string;
  section: string;           // CHALLENGE 6: this room's section in
                             rooms_pack.json
  background: string;       // CHALLENGE 6: keys, not files - the pack says
                             which file each key is
  props: RoomProp[];
  sound: string;            // played as you walk in
}
```

```
// CHALLENGE 6: the key the pack file itself is kept under, once loaded for
this room
export function packKey(room: Room): string {
  return `pack_${room.section}`;
}
```

src/scenes/MenuScene.ts

```
private queueRoom(room: Room): void {
  this.load.pack(packKey(room), ROOMS_PACK_FILE, room.section);
}
```

```
// CHALLENGE 6: the pack file itself is kept in the JSON cache, under its  
key. Left there,  
// the loader would skip it next time - and so load none of the room's files  
this.cache.json.remove(packKey(room));
```

`isLoading()` now checks the background, every prop's key, and the sound key - the keys the room actually uses.

The trap, and the reason for the last hint: a loaded pack is stored in `this.cache.json` under the key it was loaded with. After U removes the pictures and sounds, loading the room again with the same pack key is **skipped** - the key is in the cache - so no files are queued, the loader completes at once, and the room starts with nothing loaded. Tested with the harness: with the `this.cache.json.remove` line taken out, going back into an unloaded room draws every picture as the missing texture and stops with `Error: Audio key "shoot" not found in cache`. With it, U and going back in works every time.

Look for:

- a different key per room for `load.pack` (one key for all four means only the first room ever loads - the others' packs are skipped)
- the pack's `"player"` entry in two sections - the loader skips it the second time, as before
- `rooms.ts` no longer containing any file names
- an alternative that is just as good: load `rooms_pack.json` once with `this.load.json`, and give `this.load.pack` one section of it as an object instead of a file name -
`this.load.pack({ key: packKey(room), url: { [room.section]:
packData[room.section] } })`

Slides

Chapter 3

Preloading

Queues, caches, loading screens - and files that go missing



1 / 20

Today

- what `this.load.image(...)` really does
- keys, and the game's **caches**
- **Boot -> Preload -> Menu**, with a progress bar
- every kind of file: image, sprite sheet, audio, JSON, text, pack
- when a file is missing
- loading in the middle of a game

2 / 20

preload() only makes a list

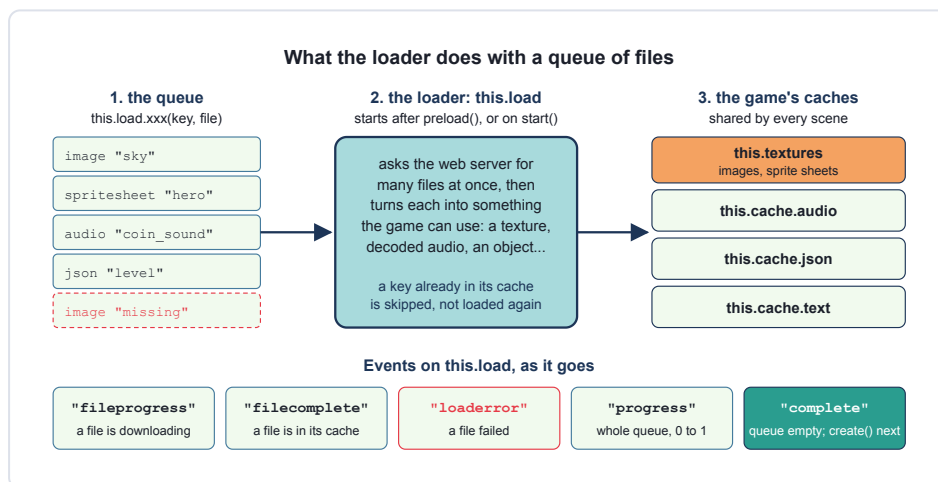
```
// preload() - queued, NOT loaded
this.load.image(CRATE_KEY, CRATE_FILE);

// create() (in showImages) - loaded by now
this.add.image(x + 70, y + 120, CRATE_KEY).setScale(2);
```

1. `preload()` queues files
2. Phaser starts the loader when `preload()` returns
3. `create()` runs when the queue is empty

3 / 20

Queue -> loader -> caches



4 / 20

Keys and caches

Loaded with	Kept in
image , spritesheet	this.textures
audio	this.cache.audio
json	this.cache.json
text	this.cache.text

- the caches belong to the **game** - every scene can use them
- a key is unique **within its kind**
- a key that is already loaded is **skipped**

5 / 20

Paths

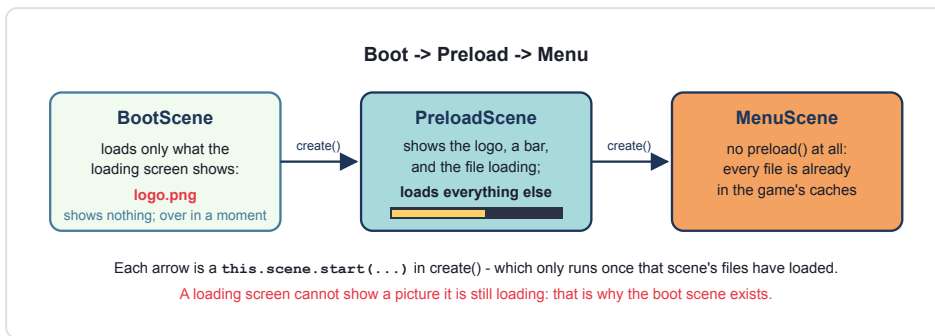
```
this.load.setPath("assets/images/");
this.load.image(SKY_KEY, "sky.png");
this.load.image(SPACE_KEY, "space.png");

this.load.setPath("assets/audio/");
this.load.audio(COIN_SOUND_KEY, "coin.wav");
```

- address = **base URL + path + file name**
- relative to the **page**: public/ is copied to dist/
- no leading / ; capitals matter on a web server

6 / 20

Boot -> Preload -> Menu



A loading screen cannot show a picture it is still loading.

7 / 20

The boot scene

```
preload(): void {
    this.load.image(LOGO_KEY, LOGO_FILE);
}

// create() only runs once everything queued in preload() has loaded
create(): void {
    this.scene.start(PRELOAD_SCENE);
}
```

Small, fast, shows nothing.

8 / 20

Drawing the bar: Graphics

```
const box = this.add.graphics();
box.fillStyle(BOX_COLOUR);
box.fillRect(barX - BOX_PADDING, BAR_Y - BOX_PADDING,
  BAR_WIDTH + BOX_PADDING * 2, BAR_HEIGHT + BOX_PADDING * 2);

const bar = this.add.graphics();
```

- `fillStyle(0x2b3245)` - colours as hex numbers
- `fillRect(x, y, width, height)` - from the **top-left**
- `clear()` - rub it all out

9 / 20

Listening to the loader

```
this.load.on("progress", (value: number) => {
  bar.clear();
  bar.fillStyle(BAR_COLOUR);
  bar.fillRect(barX, BAR_Y, BAR_WIDTH * value, BAR_HEIGHT);
});

this.load.on("fileprogress", (file: Phaser.Loader.File) => {
  fileText.setText(`Loading ${file.src}`);
});

this.load.on("complete", () => {
  fileText.setText("All done!");
});
```

Also: `"start"`, `"filecomplete"`, `"loaderror"`

10 / 20

Try it: Loading Screen

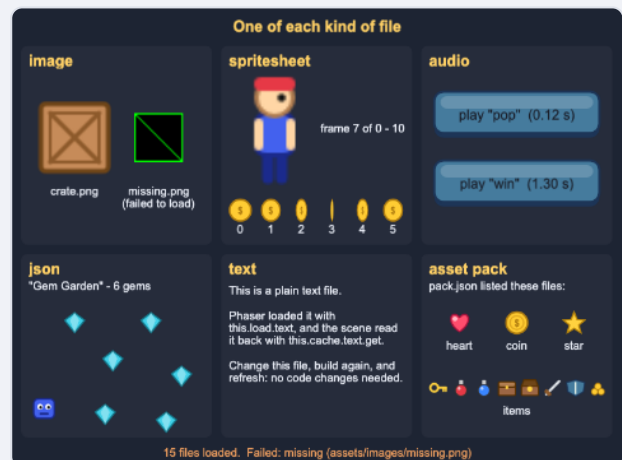
- build and run `ch03_loading_screen` - a flash
- F12 -> **Network** -> **Disable cache**, **Fast 4G** -> refresh
- set `COPIES` to 0 - what changes?
- press **R** in the menu: "Loaded **0** files" - why?



11 / 20

Every kind of file

```
this.load.image(CRATE_KEY, CRATE_FILE);
this.load.spritesheet(HERO_KEY, HERO_FILE, {
  frameWidth: 32, frameHeight: 48 });
this.load.audio(POP_KEY, POP_FILE);
this.load.json(LEVEL_KEY, LEVEL_FILE);
this.load.text(NOTES_KEY, NOTES_FILE);
this.load.pack(PACK_KEY, PACK_FILE,
  PACK_SECTION);
```



12 / 20

Sprite sheets

- one picture, cut into equal **frames**, numbered from 0
- `coin_spin.png` is 192 x 32 -> six 32 x 32 frames
- the frame is the fourth argument:

```
this.add.image(frameX, y + 215, COIN_SPIN_KEY, frame);
```

- wrong frame size = frames cut in the wrong place
- no frame size at all = `Invalid frameWidth given.`

13 / 20

JSON - and saying what is in it

```
export interface LevelData {  
  name: string;  
  player: Point;  
  gems: Point[];  
}  
  
const level: LevelData = this.cache.json.get(LEVEL_KEY);  
level.gems.forEach((gem) => {  
  this.add.image(originX + gem.x, originY + gem.y, GEM_KEY);  
});
```

`get` returns `any`. The interface checks the **code**, never the **file**.

14 / 20

Asset packs

```
{
  "pickups": {
    "path": "assets/",
    "files": [
      { "type": "image", "key": "heart", "url": "images/heart.png" },
      { "type": "image", "key": "coin", "url": "images/coin.png" },
      ...
    ]
  }
}
```

- the list of files moves **out of the code**
- sections; each can set its own `path`
- the pack is a file too - with a key, in the JSON cache

15 / 20

When a file is missing

```
this.load.on("loaderror", (file: Phaser.Loader.File) => {
  this.failed.push(`${file.key} (${file.src})`);
});
```

Missing	Result
picture	the missing texture - black, green outline
sound	Audio key "pop" not found in cache
JSON / text	undefined

First stop: the console's red **404** line.

16 / 20

Loading later

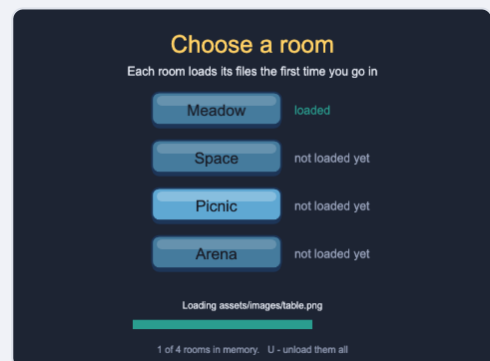
```
this.queueRoom(room);
this.load.once("complete", () => {
    this.enterRoom(index);
});
this.load.start();
```

- outside `preload()` , **you** start the loader
- check first: `this.textures.exists(key)` , `this.cache.audio.exists(key)`
- ignore clicks while `this.load.isLoading()`
- let go: `this.textures.remove(key)` , `this.cache.audio.remove(key)`

17 / 20

Try it: Load on Demand

- throttle the network, choose a room: watch it load
- ESC, choose it again: instant
- Meadow, then Arena: one file fewer - why?
- U, then go back in



18 / 20

Summary

- `load... queues`; `create()` runs when the queue is empty
- files are kept **by key**, in game-wide caches; loaded keys are skipped
- Boot -> Preload -> Menu; a `Graphics` bar moved by `"progress"`
- image, spritesheet, audio, json (+ interface), text, pack
- missing: `"loaderror"`, the missing texture, the console
- later: queue, `once("complete")`, `this.load.start()`

19 / 20

Challenges

1. **Restyle the bar** - rounded, outlined, red -> orange -> green
2. **Percentage** - whole numbers, never 100% too soon
3. **Hearts in the level** - JSON and interface together
4. **Keep it moving** - a spinner that turns while loading
5. **When files fail** - an error screen, try again or carry on
6. **Rooms from a pack** - every room's files in one asset pack

Next: **Chapter 4 - The life of a scene**

20 / 20